MSAGLJS: Graph Layout, Edge Routing, and Tiling for the Web

Anonymous author(s)
Anonymous affiliation(s)

Abstract

We present MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, distributed as a set of NPM packages[1]. It provides a set of layout algorithms, a set of edge routing algorithms, and two renderers that run in an Internet browser: one based on SVG and another based on WebGL via the deck.gl framework. In this paper we concentrate on two features of the latter renderer.

First, a *tiling scheme* that turns a large graph into a navigable map with *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on digital maps—making the browsing experience familiar to anyone used to online maps. Second, a *corridor routing* method that routes each edge along the shortest path in homotopy class around the node obstacles, based on the Constrained Delaunay Triangulation. The whole pipeline runs client-side, without a dedicated server, on a phone or a laptop. In this configuration the WebGL renderer interacts smoothly with graphs of up to 10 000 nodes and 40 000 edges.Re-run benchmarks to confirm the maximum graph size; current 10 000 / 40 000 figures are placeholders.

MSAGLJS is available at `https://github.com/microsoft/msagljs`.

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,[2] and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS[3] is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme for level-of-detail rendering.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [4]. It supports the layered Sugiyama scheme [42], Pivot MDS [16], and IPSep-CoLa [22], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogous

[1] `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-webgl` — all available on `https://www.npmjs.com/`.

[2] In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2^{32} bytes).

[3] `https://github.com/microsoft/msagljs`

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).
Editors: TBD; Article No. 0; pp. 0:1–0:14
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

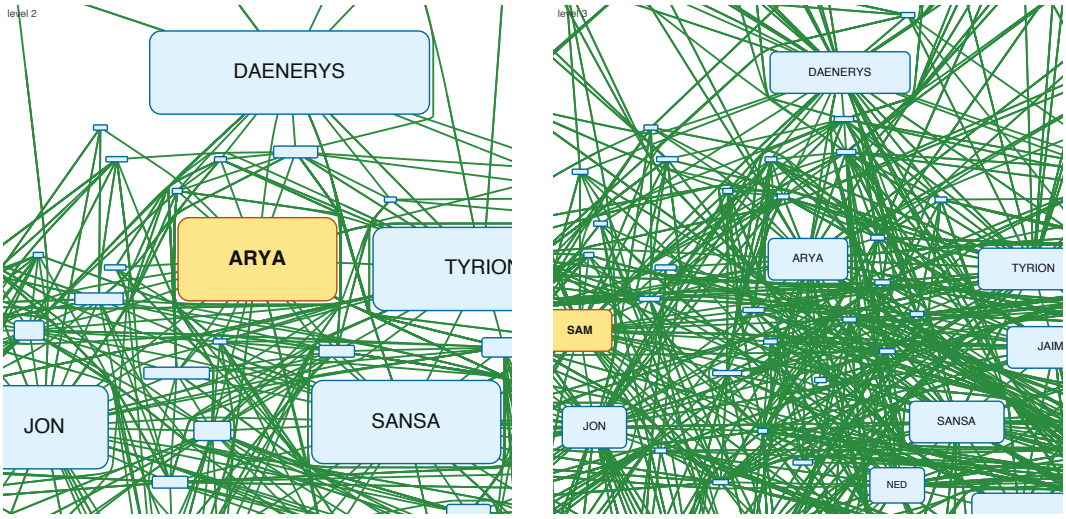

Figure 1 Overview of the per-level-graph tiling scheme presented in Section 4, on the Game of Thrones graph. Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes, and the top-ranked landmark, highlighted in amber, enters that level enlarged by a factor of two in each dimension. Remaining survivors, shown in light blue, are kept at a node-specific scale $s_v \in [0.25, 2]$ that shrinks with rank so that they do not overlap. Edges are rerouted per level with a sleeve-hint A* built from the finer-level route, which keeps the coarse shape close to the fine one and yields smooth transitions as the user pans and zooms.

to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers` package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs[4] and an SVG renderer for smaller, editable graphs[5].

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *corridor routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

Graphviz [26, 6] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [9]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [38] that did not have the features described here. On the web, Sigma.js [10] and Cytoscape.js [24] provide interactive graph rendering, but rely on straight-line or generic

[4] `https://microsoft.github.io/msagljs/renderer-webgl/index.html`
[5] `https://microsoft.github.io/msagljs/renderer-svg/index.html`

spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [8] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [3] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [37] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [40] handles huge graphs with tiles and edge bundling [35, 33] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [14], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [25], which distort a multilevel layout [28, 34] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [32] and force-directed bundling [33] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our corridor algorithm walks the dual of a Constrained Delaunay Triangulation [41] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [36, 17, 27, 5]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [31] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [21] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [19] but requires a simple containing polygon. yWorks [13] offers "Organic edge routing" based on force-directed simulation. The approach of Dwyer and Nachmanson [23] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [43]. Our corridor method eliminates the spanner entirely and routes directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

3 System Architecture

MSAGLJS is organized as a monorepo with five NPM packages:

- `@msagl/core` — Graph data structures, layout algorithms (Sugiyama, MDS, IPSep-CoLa), edge routing, CDT, tiling.
- `@msagl/drawing` — Visual attributes (colors, shapes, labels) bridging the graph model and renderers.
- `@msagl/parser` — Parsers for DOT and JSON graph formats.
- `@msagl/renderer-svg` — SVG-based renderer for small graphs.
- `@msagl/renderer-webgl` — WebGL renderer using deck.gl [4], with tiling support.

The layout pipeline proceeds in three phases:

1. **Layout.** Node positions are computed by the chosen algorithm. For large undirected graphs the default is force-directed layout (IPsepCola) initialized with Pivot MDS [16]; for directed graphs, Sugiyama layout is used.
2. **Edge routing.** Edges are routed around node obstacles. The routing mode is configurable; this paper focuses on *corridor routing* (Section 5).
3. **Tiling.** A tile hierarchy is built for level-of-detail rendering (Section 4).

The WebGL renderer (Section 7) consumes the tile hierarchy and renders the appropriate level of detail based on the current viewport.

4 Tiling

To visualize large graphs interactively, MSAGLJS builds a pyramid of tiles analogous to those used in online maps, served to the browser through the `TileLayer` of the `@deck.gl/geo-layers` package [12]. Levels are numbered $0, 1, \ldots, Z$ from coarsest to finest; level 0 is a single tile covering the whole drawing with only a few top-ranking graph features, and the finest level Z corresponds to the full graph produced by the layout and the corridor router of Section 5.

Conceptually, each tile is associated with a rectangular viewport and holds the graph elements that lie within it: the single tile at level 0 is associated with a viewport covering the entire drawing, so it contains the whole graph. When rendering, every level shows only a prefix of the graph elements ranked by importance, and each tile renders the part of that prefix that lies within its viewport; the remaining elements are hidden. The per-tile capacity $\mathcal{C}$ only governs the choice of Z (when to stop subdividing) and is not consulted when deciding what to render at each level: the selection of visible elements per level is driven entirely by importance and geometric non-overlap (Section 4.1). Only the finest level Z displays every node and edge of the graph.

Per-tile rendering load. At the finest level, condition (i) is intended to cap each tile at $\mathcal{C}$ visible elements (nodes plus edge clips, default $\mathcal{C} = 500$). On dense graphs, however, either the depth cap $Z_{\max} = 8$ or the minimum tile size (ii) binds first, in which case the densest finest-level tile may hold many more than $\mathcal{C}$ elements. For a coarser level $z < Z$ we ignore $\mathcal{C}$ when preparing a tile for rendering. We apply semantic zoom on the nodes, (Section 4.1) by inflating each surviving node by roughly 2^{Z-z} in linear size. The chosen nodes define the rendered edges: we render the edges connecting two chosen nodes. Table 1 reports the empirical worst-case per-tile element count on a range of graphs.

4.0.0.1 How Z is chosen.

Z is determined incrementally rather than fixed in advance. We initialize $Z = 0$ with the single root tile covering the whole drawing. While the tiles at level Z violate conditions (i) or (ii) below, we increase Z by one and split every tile of the previous finest level into four equal sub-tiles. The two conditions are: (i) every tile contains at most $\mathcal{C}$ visible elements (default $\mathcal{C} = 500$), preventing overloaded tiles; and (ii) the tile width and height are both below ten times the average node width and height, preventing tiles smaller than the nodes they display. We additionally enforce a hard cap $Z \le Z_{\max}$ derived from a memory estimate. At level z the pyramid has a $2^z \times 2^z$ grid of tiles, so the finest grid resolution is $K = 2^Z$. Across the whole pyramid we estimate the total number of stored tile elements as

$$T(K) \;\approx\; 2\,\big((K/4)\,M + N\big),$$

where N and M are the node and edge counts of G. The term $(K/4)\,M$ counts per-tile edge clips at the finest level, under the heuristic that an average edge crosses $K/4$ tiles. The additive N counts node copies. The factor 2 accounts for the contribution of all coarser levels combined. With roughly 200 bytes per stored element and a maximum memory $\mathcal{M}_{\max}$ (default 4 GB), we solve $200\,T(K) \le \mathcal{M}_{\max}$ for the largest admissible K,

$$K_{\max} \;=\; \Big\lfloor\, 4\big(\mathcal{M}_{\max}/(2 \cdot 200) - N\big)/M \,\Big\rfloor,$$

Algorithm 1 BUILDTILEPYRAMID($G, \mathcal{C}, minTileSize$)

$T_0 \leftarrow$ single tile holding all of G clipped to its bounding box
$levels \leftarrow [\,T_0\,]$; $z \leftarrow 0$
while some tile in $levels[z]$ exceeds $\mathcal{C}$ elements **and** its width/height $\geq minTileSize$ **do**
    $levels[z{+}1] \leftarrow$ split every tile of $levels[z]$ into four sub-tiles
    within each parent, distribute nodes/labels/arrowheads by bbox, and split each edge clip at the tile midlines (Section 4.3)
    $z \leftarrow z + 1$
end while
$Z \leftarrow z$; $V_Z \leftarrow V(G)$; $s_v^{(Z)} \leftarrow 1$ for all $v \in V_Z$
compute PageRank(G)
for $z = Z - 1$ **down to** 0 **do**
    $(V_z, \{s_v^{(z)}\}) \leftarrow$ SELECTTOPKWITHADAPTIVESCALE(V_{z+1}, 2^{Z-z}) ▷ Section 4.1
    build a CDT from the inflated polylines of V_z at scales $\{s_v^{(z)}\}$
    reroute every edge $e \in E_z$ on this CDT, using the level-($z{+}1$) route as a sleeve hint ▷ Section 4.2
    for every tile t at level z, recompute the clips of e inside t from the new curve, splitting at tile midlines
end for
return *levels*

and set

$$Z_{\max} \;=\; \min\bigl(8,\; \lfloor \log_2 K_{\max} \rfloor\bigr).$$

The constant 8 is a safety ceiling: even when $\mathcal{M}_{\max}$ allows a much larger grid, we never build more than nine pyramid levels.

4.0.0.2 Three phases.

Once Z and the tile rectangles are fixed, construction proceeds in three phases for every coarser level $z = Z{-}1, Z{-}2, \ldots, 0$: (1) use PageRank to pick the nodes shown at level z and possibly enlarge them (Section 4.1); (2) reroute the edges of G_z around these obstacles, which may now be enlarged, (Section 4.2); (3) recompute the per-tile edge clips at level z by splitting the new curves at tile boundaries (Section 4.3). Figure 1 on the first page shows two adjacent pyramid levels built by this procedure. Algorithm 1 summarizes the whole pipeline.

4.1 PageRank-Guided Level Construction

To keep labels readable at coarse zooms, each level $0 \leq z < Z$ is assigned its own *level graph* $G_z = (V_z, E_z)$: a subset $V_z \subseteq V_{z+1}$ of active nodes together with every edge of G whose endpoints both lie in V_z. We compute PageRank [39] on G once, sort all nodes by descending PageRank as $v_0, v_1, \ldots, v_{|V|-1}$, and build the levels top-down starting from $V_Z = V$. Write $S_z = 2^{Z-z}$ for the per-level scale ceiling.

4.1.0.1 Building V_z.

The intuition is a greedy seniority sweep: highly ranked nodes get to "claim" as much screen room as the level allows, and lower-ranked ones squeeze into whatever space is left.

Concretely, the candidate set at level z is the rank-prefix

$$T_z \;=\; \{\, v_0, v_1, \ldots, v_{n_z-1} \,\}, \qquad n_z = \lceil |V|/2^{\,Z-z} \rceil,$$

i.e. a fixed-size prefix of the *globally* rank-sorted node list, halving in size with each coarser level. Working from a global prefix rather than from the previously built V_{z+1} matters: a high-rank node that was rejected once because of a near-by even-higher-rank neighbor at a finer level would otherwise be permanently lost, even at coarser levels where its enlarged box would actually fit. We process T_z in rank-descending order. Each candidate v is assigned the largest scale $s_v \le S_z$ at which its bounding box, scaled about v's center and inflated by a margin m that absorbs the router's obstacle padding and the CDT inflation, stays disjoint from the inflated boxes of every already-accepted node. Disjointness is a closed-form, axis-aligned separating-axis test, so $s^\star(v)$ is obtained directly from box centers and dimensions; if $s^\star(v) \ge 1$ the candidate is accepted at $s_v = \min(\sigma, s^\star(v))$, where the running cap σ (initially S_z) is then tightened to s_v to forbid any later, lower-ranked node from being drawn larger than its accepted seniors. Because v_0 is processed first against an empty accepted set, it always survives at the full scale S_z. The accepted set becomes V_z and the per-node scales $\{s_v^{(z)}\}$ are stored for the level. Because $T_z \subseteq T_{z+1}$ and at coarser levels boxes are only larger (so an already-tight overlap configuration cannot become looser), acceptance is monotone in z: $V_z \subseteq V_{z+1}$, as required by the level-graph definition. The result is the intended map-like effect: a few large landmarks at coarse zooms, progressively more detail as the user zooms in, and a guaranteed visible gap of at least $2m$ between every pair of un-inflated scaled boxes.

To avoid testing every candidate against every previously accepted box, accepted boxes are maintained in a uniform spatial hash whose cell size is an upper bound on the inflated box dimension at the current level. Two boxes can overlap only if their center cells are within one step on each axis, so each candidate consults at most nine cells. On the layouts we tested the expected work per candidate is $O(1)$, giving an overall expected cost of $O(|T_z| \log |T_z|)$ dominated by the initial sort; the worst case, when many boxes pile into a single cell, degrades to $O(|T_z| \cdot |V_z|)$.

4.2 Stable Per-Level Rerouting with a Routing Hint

Because G_{z-1} has fewer obstacles than G_z, and different ones on account of the enlargement of v_0, edges of G_{z-1} must be rerouted: the finest-level curves would otherwise pierce the enlarged landmark boxes. We rerun the corridor router (Section 5) on the active edges of G_{z-1}, but with two modifications that secure visual stability during zoom.

4.2.0.1 Per-level CDT.

For level $z-1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using each node's level-$z{-}1$ scale. The global landmark v_0 thus enters the triangulation as an obstacle that is $2^{Z-z+1}\times$ its finest-level size in each dimension, so routes naturally bend around it.

4.2.0.2 Routing hint.

For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z. We sample c_e, locate the CDT triangles it visits at level $z-1$, and expand this sequence by one neighbor hop to obtain a triangle *sleeve* S_e. Instead of the per-source Dijkstra trees of Section 5.4, which are efficient when many edges share a source but ignore the previous

level's shape, we run a per-edge **A*** on the dual of the level-$z-1$ CDT, *restricted to* S_e, with the source and target obstacle triangles always admissible. The hint keeps the coarse-level route topologically and geometrically close to the fine-level one, so that as the user zooms out an edge deforms smoothly rather than jumping to a different side of a landmark. If the masked A* fails, for instance when the target now lies inside the inflated landmark, we fall back to unconstrained A* on the whole dual, and finally to A* with the obstacle-interior guard disabled; all three tiers share the same pre-allocated open-set arrays.

To summarize the division of labor: **Dijkstra trees**, Section 5.4, drive the *finest-level* routing, where there is no hint and many edges share sources; **A*** drives the *per-level rerouting* for $z < Z$, where each edge carries its own sleeve hint from the finer level and independent searches are required.

4.3 Splitting Edges between Tiles

Once each level G_z has its final geometry, we cut that level into tiles. A tile is a pair $(\mathit{rect}, \mathit{tiledata})$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair (e, p), with p a continuous piece of the edge curve c_e confined to *rect*.

The single tile at level 0 is obtained by clipping every element to the graph's bounding box. To build level $z+1$ from level z, each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines, producing sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile's rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge e inside a tile equals $c_e \cap \mathit{rect}$. Invariant $\mathcal{F}$ serves two purposes. Part (a) makes subdivision cheap and local. Suppose a parent tile has rectangle *rect* with horizontal and vertical midlines ℓ_h, ℓ_v. To split a clip (e, p) into its children, we collect all intersections of the underlying curve c_e with ℓ_h and ℓ_v, restrict them to the parameter range $[\mathit{start}, \mathit{end}]$ defining p, sort these parameters, and prepend *start* and append *end*. This yields a sorted parameter list $\mathit{start} = t_0 < t_1 < \cdots < t_m = \mathit{end}$, and the m child sub-clips are exactly the curve pieces over consecutive intervals $[t_u, t_{u+1}]$. Each piece is assigned to one of the four children by sampling c_e at the midpoint $(t_u + t_{u+1})/2$ and comparing against the midlines. Crucially, no intersection with the four outer sides of *rect* is ever computed: invariant (a) guarantees that the curve already lies inside *rect* between t_0 and t_m and crosses its boundary only at those endpoints, so the midlines alone fully decompose the clip into rectangle-confined pieces. This reduces subdivision to one curve–line intersection per midline plus a sort, instead of a full 2D rectangle clip. Part (b) is the rendering-correctness condition: at any zoom only the tiles intersecting the viewport are fetched and their clips drawn, and (b) guarantees that the union of what is drawn equals $c_e \cap$ viewport for every visible edge e, so no segment is missed and none is drawn twice. The same invariant is also what makes the rerouting of Section 4.2 compatible with tiling: whenever an edge curve c_e is replaced by a coarser-level route, (b) is reestablished by recomputing the clips of that edge inside every affected tile before the level is served.

Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. These are the same conditions that determine Z (Algorithm 1).

Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

Table 1 Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$). The capacity $\mathcal{C}$ only guides the choice of Z; we have no analytical bound on the actual per-tile element count, which is therefore reported empirically. "Max tile's level" is the level on which the maximum tile was found. In the worst cases (`facebook_combined`, `ca-CondMat`) the densest tile holds 20–30× the nominal capacity $\mathcal{C}$.

Graph	$\lvert V\rvert$	$\lvert E\rvert$	Levels built	Max per tile	Max tile's level
gameofthrones	407	2 639	5	1 168	4
composers	3 405	13 832	8	3 317	7
ca-GrQc	5 241	14 484	8	934	7
ca-HepTh	9 875	25 973	9	1 598	8
facebook_combined	4 039	88 234	9	11 874	8
ca-CondMat	23 133	186 878	9	13 744	8
deezer_europe	28 281	92 752	9	8 201	8
delaunay_n15	32 768	98 274	9	611	8

5 Corridor Routing on the CDT

We describe an edge routing method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a *corridor*—a sequence of adjacent CDT triangles from source to target—and applies the funnel algorithm to produce the shortest path within it.

5.1 CDT Construction and the Dual Graph

Given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [18, 20]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. A triangle is *free-space* if it is not contained in any obstacle interior. Each triangle of an A* (or Dijkstra) path is entered through one shared edge and left through another; the cost of the step taken inside the triangle is the Euclidean distance between the midpoints of those two edges. For the source triangle, which has no entering edge, the centroid is used as the entry point.

5.2 A* Search on the Dual Graph

Intuitively, the CDT partitions free space into triangular "rooms" connected by shared edges acting as "doorways". Routing an edge from s to t then splits into two subproblems: a *combinatorial* one—which sequence of rooms to traverse—and a *geometric* one—which exact polyline through that sequence is shortest. We solve the first with a best-first search on the dual graph, and the second with the classical funnel algorithm. The dual-graph search ignores fine geometry and only commits to a corridor; the funnel then pulls the path taut inside that fixed corridor, sliding it against diagonal endpoints until it is locally straight. Because A* on the dual uses centroid-to-centroid step costs and a straight-line heuristic to the target, it is consistent and returns a corridor whose total cost lower-bounds, but closely tracks, the true shortest path—good enough to feed the funnel without chasing exact geometry inside the search.

Concretely, for each graph edge from source s to target t, we locate the containing triangles and run A* [29] on the dual graph D. The heuristic is the straight-line distance

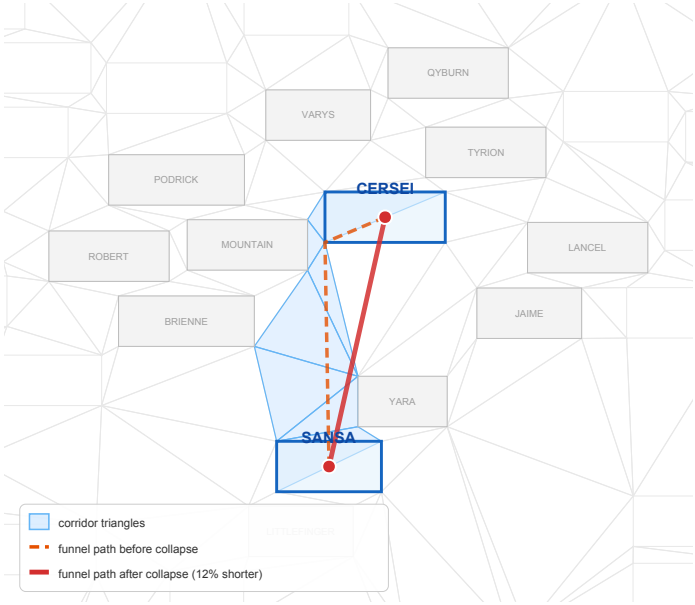

Figure 2 Effect of vertex collapse on the CERSEI–SANSA edge. Light blue: corridor triangles. Orange dashed: funnel path before collapse (3 waypoints). Red solid: funnel path after collapse (2 waypoints, 12% shorter). Collapsing obstacle vertices to the node center widens the funnel opening and removes the detour.

from the current centroid to t. Triangles inside obstacles (other than the source and target obstacles) are excluded from the search. The resulting corridor is passed to the funnel algorithm [17, 30], which computes the shortest path through the corridor's sequence of diagonals.

5.3 Vertex Collapse at Source and Target

The corridor passes through triangles whose vertices lie on the padded obstacle boundaries. When these vertices appear as diagonal endpoints, the funnel algorithm produces sharp zigzag turns near each endpoint. We eliminate these artifacts by *collapsing* obstacle vertices: any diagonal endpoint belonging to the source obstacle is replaced with the source center s, and similarly for the target. Degenerate diagonals (both endpoints collapsed to the same point) are removed. The left/right orientation of each diagonal is preserved from the uncollapsed geometry.

After collapse, the funnel sees a wide opening at each endpoint and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves.

5.4 Batched Routing with Dijkstra Trees

When a node has many incident edges, running independent A* searches is wasteful. We group edges by source node and run a single Dijkstra computation per source on the dual graph, expanding until all target triangles are reached. Corridors are recovered by following parent pointers. This reduces the number of searches from m (edges) to at most n (nodes). On the Game of Thrones social network (407 nodes, 2 639 edges, 331 distinct sources), the Dijkstra tree approach reduces routing time by 30%.

6 The Portal Graph

The batched Dijkstra approach of Section 5.4 still requires exploring much of the CDT for each source node. To enable faster queries, we introduce the *portal graph*—an abstraction that separates the free-space routing problem from the local obstacle traversal.

6.1 Portal Triangles

For each node obstacle, we identify its *portal triangles*: the free-space CDT triangles that share an edge with the obstacle boundary. Formally, a triangle t is a portal of obstacle O if t is free-space and at least two of its three CDT sites belong to O's boundary polyline. Portals serve as entry and exit points: any shortest path from the interior of O to free space must pass through a portal triangle.

6.2 Structure of the Portal Graph

The portal graph G_P is defined as follows:

- **Nodes.** For each graph node v, the center point c_v and all portal triangles $\{p_1^v, p_2^v, \ldots\}$ of v's obstacle are nodes of G_P.
- **Interior edges.** Each center c_v is connected to every portal p_i^v of the same obstacle, with weight equal to the BFS distance through the obstacle's interior triangles.
- **Free-space edges.** Portal triangles of different obstacles are connected through the free-space CDT dual graph.

To route a graph edge from node w to node u, we find the shortest path in G_P from c_w to c_u. This path decomposes into three segments:

1. An *interior path* from c_w through w's obstacle to some portal p_i^w.
2. A *free-space path* from p_i^w to some portal p_j^u of u.
3. An *interior path* from p_j^u through u's obstacle to c_u.

The free-space segment (2) is the bottleneck: it requires a shortest-path query on the free-space dual graph, which may contain thousands of triangles. We answer these queries with the batched Dijkstra trees of Section 5.4.

7 WebGL Rendering

The WebGL renderer uses deck.gl [4], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView` (no perspective distortion) and a `TileLayer` that dynamically loads tile data based on the current viewport.

7.0.0.1 Tile resolution mapping.

The root tile size is rounded up to the nearest power of two. A *start zoom* level is computed so that the root tile maps to the standard 512-pixel tile size of deck.gl. The `TileLayer`'s `getTileData` callback translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the precomputed tile data.

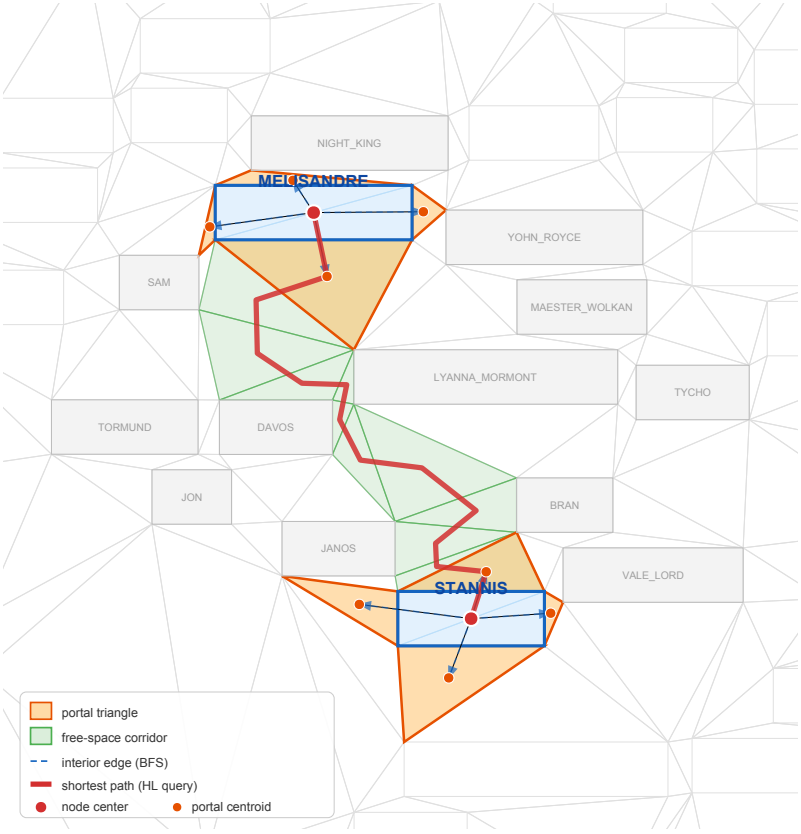

Figure 3 The portal graph for the MELISANDRE–STANNIS edge on the Game of Thrones social network. Orange triangles are portals—free-space CDT triangles adjacent to the obstacle boundary. Blue dashed arrows show interior edges from node centers to portal centroids. Green-shaded triangles form the free-space corridor found by the batched Dijkstra search. The thick red polyline shows the shortest path $c_{\text{MEL}} \to p_i \to \cdots \to p_j \to c_{\text{STAN}}$. Nearby node obstacles are shown in gray.

7.0.0.2 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level z, the resolution is 2^{z-2} segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

7.0.0.3 Style-based filtering.

A declarative style specification allows per-layer control of visibility ranges (`minZoom`, `maxZoom`), colors, opacity, sizes, and filters. Styles can reference the node's PageRank to, e.g., show labels only for important nodes at medium zoom levels.

Graph	Nodes	Edges	CDT △	Free △	Portals	Dijkstra (ms)
GoT [15]	407	2 639	3 258	2 444	1 628	277
b103 [2]	944	2 438	7 554	5 666	3 776	754
b100 [1]	1 463	5 806	11 706	8 780	5 852	2 120
composers [11]	3 405	13 832	27 242	20 432	13 620	9 335
Gnutella04 [7]	10 876	39 994	87 010	65 258	43 504	93 760

Table 2 Corridor routing benchmarks. Dijkstra: per-source Dijkstra trees on the CDT dual.

7.0.0.4 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

8 Experimental Results

We evaluate the corridor routing method on benchmark graphs from the Game of Thrones social network [15], the GD 2011 contest [11], the Graphviz test suite [2, 1], and the Gnutella P2P network [7]. All experiments run in Node.js on a laptop with an Apple M1 processor. Layout uses Pivot MDS; timings include CDT construction, routing, and curve smoothing.

Table 2 reports the performance of the Dijkstra-tree corridor router (Section 5.4). For each graph, we report the number of CDT triangles, free-space triangles, the total number of portal triangles across all nodes, and the total routing time.

The Dijkstra-tree approach scales to graphs with tens of thousands of edges. Routing the full Gnutella04 graph (10 876 nodes, 39 994 edges) on an Apple M1 laptop takes about 94 s, which is amortized in interactive use because tile pyramids cache the routes computed at the finest level and rerun only the per-level A* described in Section 4 on coarser zooms.

9 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The corridor routing method routes edges directly on the CDT dual graph using a portal graph abstraction and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at `https://github.com/microsoft/msagljs` under the MIT license.

References

1 b100. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b100.dot`.

2 b103. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b103.dot`.

3 Cosmograph. `https://cosmograph.app`.

4 deck.gl: Large-scale WebGL-powered data visualization. `https://deck.gl/`.

5 Funnel algorithm. `https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf`.

6 Graphviz. `http://www.graphviz.org/`.

7 p2p-gnutella04. `https://snap.stanford.edu/data/p2p-Gnutella04.html`.

8 Regraph. https://cambridge-intelligence.com/regraph/.

9 sfdp. https://graphviz.org/docs/layouts/sfdp/.

10 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. https://www.sigmajs.org/.

11 Skewed. http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml.

12 TileLayer (@deck.gl/geo-layers). https://deck.gl/docs/api-reference/geo-layers/tile-layer. Accessed: 2026.

13 yworks. https://yworks.com/products/yed.

14 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.

15 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.

16 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18-20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.

17 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.

18 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.

19 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.

20 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.

21 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. https://arxiv.org/abs/2504.17033.

22 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.

23 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.

24 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.

25 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005.

26 Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.

27 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987.

28 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.

29 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.

30 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.

31 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.

32 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

33 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.

34 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005.

35 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library, 2012.

36 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984.

37 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd, and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.

38 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.

39 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.

40 Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.

41 Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation. *Computational Geometry*, 22(1–3):21–74, 2002.

42 Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.

43 Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture Notes in Computer Science*, 5849:219–231, 2010.